

Introduction

Goal: Refactoring dello Sprint1 usando i protoattori

Requirements

- 1. realizzare un sistema che calcoli i valori dell'espressione $\sin(x) + \cos(\sqrt{3} * x)$ dato un input decimale x
- 2. utilizzare il [javalin server](#) per abilitare la ricezione di messaggi via WebSocket, ma anche HTTP/1 e HTTP2
- 3. devono essere accettate anche richieste di calcolo tramite MQTT per supportare dispositivi IoT e interazioni asincrone tramite broker
- 4. la logica di business deve essere isolata in un attore che estende [AbstractProtoactor26](#) presente nel progetto *protoactor26*
- 5. il sistema deve distinguere tra request (a cui risponde con una reply contenente il valore calcolato), dispatch ed event
- 6. i messaggi inviati a un singolo attore devono essere elaborati in modo sequenziale (politica FIFO) tramite una coda interna
- 7. il deployment dell'applicazione deve avvenire mediante Docker

Requirement analysis

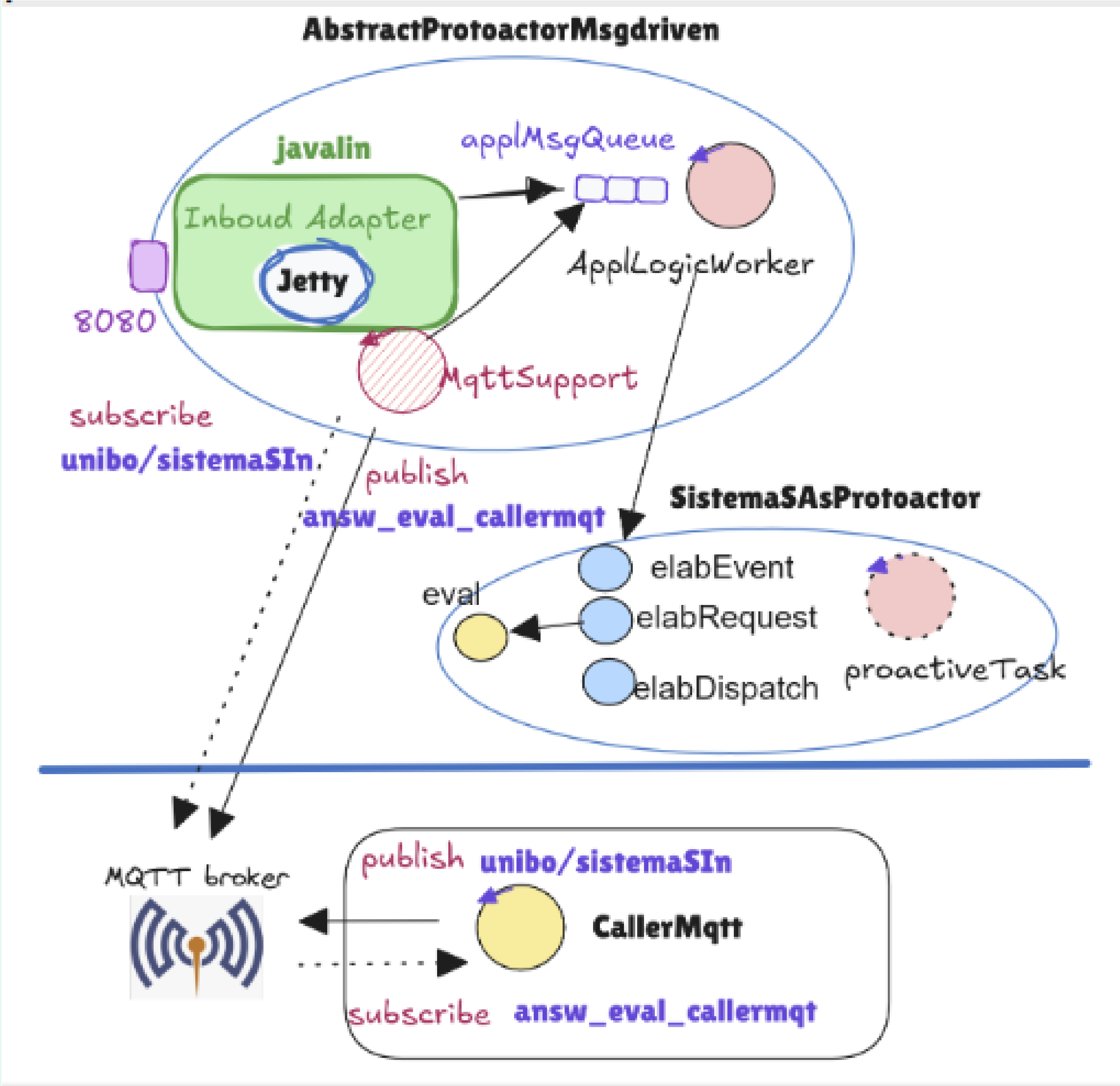
La realizzazione della funzione per calcolare il risultato dell'espressione data in Java è banale.

```
protected double eval(double x) {
    return Math.sin(x) + Math.cos( Math.sqrt(3)*x);
}
```

MQTT è un protocollo diverso da HTTP e nasce con l'idea che le informazioni emesse (published) su una topic da un componente software sono eventi che qualche altro componente (i subscribers) può percepire ed elaborare.

Problem analysis

L'architettura del progetto che voglio realizzare è la seguente:



L'idea è quella di introdurre un componente (AbstractProtoactorMsgdriven) che sia intrinsecamente capace di ricevere messaggi via rete e sia dotato di una struttura interna che pone in foreground la logica applicativa lasciando nascosti in background i diversi meccanismi di comunicazione via rete e sia anche capace di comportamenti autonomi (non legati a nessun messaggio). La classe AbstractProtoactorMsgdriven costituisce una sorta di Micro-framework che usa l'inversione di controllo per attivare operazioni applicative di gestione dei messaggi ricevuti via HTTP, WS e MQTT e posti nella coda del worker applicativo.

I [Java Executors](#) sono dei supporti utili per la realizzazione implicita della coppia applMsgQueue e ApplLogicWorker. Infatti, permettono di creare "worker" che rimangono in attesa dei task che gli sono accodati mediante il metodo submit (poiché creati come SingleThread si ha la garanzia che due task non interferiscano tra loro). Il progetto [protoactor26](#) mostra come si può andare a colmare questo abstraction gap con protoattori che usano i JavaExecutors.

Sulla base di ciò, per andare a fare refactoring dello Sprint1, andiamo a gestire in modo implicito il passaggio dal livello logico al livello realizzativo. Realizzo come protoattore SistemaSAsProtoactor, che conterrà la logica di business (la funzione eval). AbstractProtoactorMsgdriven sarà invece SistemaSJavalinBetter rafattorizzato con un riferimento al contesto su cui si scambiano i messaggi, che sfrutterà per andare a inviare le request dove prima faceva eval(x).

La classe CallerMqtt serve per andare a testare se il comportamento è quello atteso.

Test plans

Project

Testing

Deployment

Maintenance

